

Offline Storage and Synchronisation

Overview & Objectives

Offline Storage and Sync for CometChat

Objective

Concept Overview

Offline Storage

Offline Sync

Expected User Experience

Business Value

Problem Statement

Current Problem in CometChat

Scenario: User Loses Internet Connectivity

Step 1: User is Online (Everything Works)

Step 2: Internet Connection is Lost

Problem #1

Step 3: User Tries to Send a Message

Problem #2

Step 4: User Closes and Reopens App While Offline

Problem #3

Step 5: Internet Returns

Current CometChat Behavior

Current Architecture

Dependency

Core Problem Statement

What Needs to Be Solved

Goal

Success Metrics

Assumptions

Technical Scope

Local Storage

Offline Messaging

Synchronization

UI Enhancements

Offline Voice & Video Calling

Administrative Operations

Full Media Offline Support

Messages

Project Phases

Phase 1 – Core Offline Experience (MVP)

Goal

Components Covered

Conversations

Message Header

Message List

Message Composer

Users

Groups

Group Members

Local Storage

Offline Messaging

Synchronization

Acceptance Criteria

Phase 2 – Advanced Offline Experience

Goal

Message Composer Enhancements

Message Interactions

Group Enhancements

Performance Enhancements

Analytics & Monitoring

Out of Scope (Future Releases)

Detailed Steps

Step 1: Load Chat Screen

Step 2: Store Messages Locally

Step 3: Sending Messages While Offline

Step 4: Offline Action Queue

Step 5: Monitor Network Status

Step 6: Synchronize Pending Actions

Step 7: Synchronize New Incoming Messages

Step 8: Conversation Synchronization

Step 9: Read Receipt Synchronization

Step 10: Attachment Synchronization

Key Development Principles

Principle 1: Local Database is the Source of Truth

Principle 2: Update Database Before UI

Principle 3: Queue All Offline Actions

Principle 4: Incremental Synchronization

Risk Area's

1. Message Duplication & Ordering Risk

2. Large Local Database Performance Risk

3. Synchronization Reliability Across Platforms

4. Increased SDK Complexity & Maintenance Cost

Summary

Overview & Objectives

What we're building and what success looks like. Include 2-3 measurable objectives.

Offline Storage & Synchronization enables CometChat applications to provide a seamless messaging experience even when network connectivity is unavailable. Conversations, messages, users, groups, and related metadata are stored locally on the device, allowing users to browse chat history and continue sending messages while offline. Any actions performed offline are queued locally and automatically synchronized with the CometChat backend once connectivity is restored. This ensures data consistency, message reliability, and a responsive user experience across web, Android, and iOS platforms.

Offline Storage and Sync for CometChat

Objective

Enhance CometChat's messaging experience by introducing Offline Storage and Sync capabilities that allow users to access recent conversations one-on-one or in groups, view message history, and compose messages that include attachments, voice notes, and mentions. Once connectivity is restored, the feature will automatically sync your locally stored data with CometChat servers, ensuring a seamless, reliable, and uninterrupted messaging experience across the CometChat platform.

Concept Overview

Offline Storage and Sync is an offline-first messaging capability that enables chat data to be stored securely on the user's device and synchronised with the server when network access becomes available.

Currently, CometChat supports real-time messaging and retrieval of missed messages after reconnection. However, users may experience limitations when connectivity is poor or unavailable. Offline Storage and Sync addresses this gap by introducing a local storage layer and synchronisation engine within the CometChat platform.

Offline Storage

Offline Storage allows the application to persist chat-related data locally on the device, including:

- Conversations and chat history
- Recent messages
- Draft messages
- Read and delivery states
- Media metadata

This enables users to:

- View previous conversations without internet access
- Access cached chat history instantly
- Continue using the chat interface during network interruptions

Offline Sync

Offline Sync ensures that any actions performed while offline are automatically synchronized with CometChat servers when connectivity returns.

The synchronization process includes:

- Uploading queued messages created while offline
- Downloading messages received during offline periods

- Updating message statuses (Pending, Sent, Delivered, Read)
- Maintaining consistency between local storage and server data

Expected User Experience

1. User loses network connectivity.
2. Existing conversations remain accessible through local storage.
3. Messages composed offline are stored locally and marked as "Pending."
4. Once connectivity is restored, the SDK automatically synchronizes pending messages and retrieves missed messages.
5. Message statuses are updated seamlessly without requiring user intervention.

Business Value

- Improved user experience in low-connectivity environments
- Higher message delivery reliability
- Reduced dependency on constant network availability
- Faster conversation loading through local caching
- Competitive parity with modern messaging platforms such as Sendbird, WhatsApp, and Telegram

Problem Statement

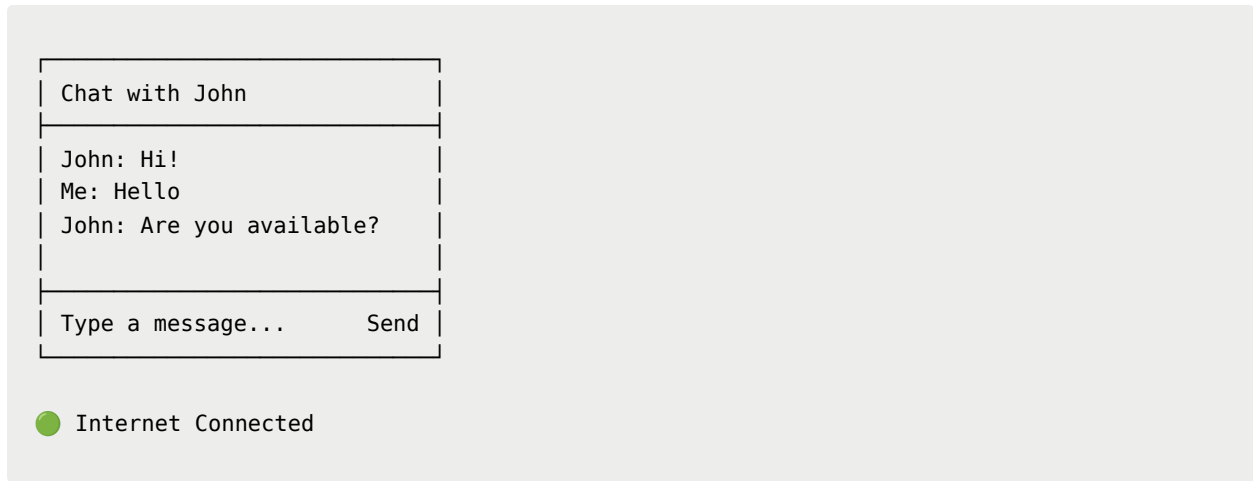
What problem are we solving, and what evidence tells us it's worth solving?

Users lose access to conversations and experience delays when network connectivity is poor or unavailable. CometChat currently supports message recovery after reconnecting but does not provide a first-class offline storage and synchronization layer.

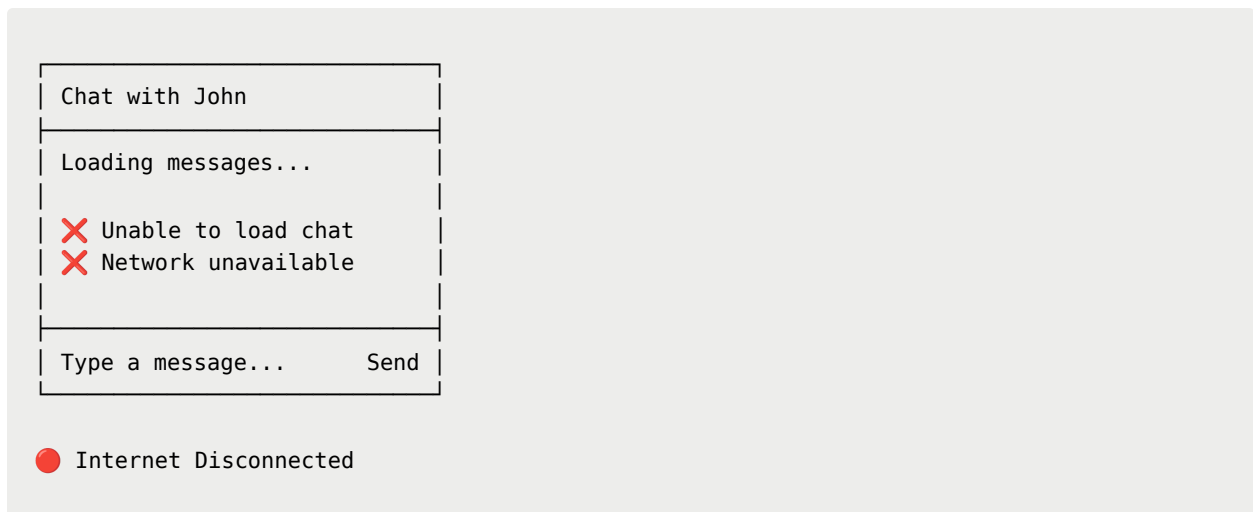
Current Problem in CometChat

Scenario: User Loses Internet Connectivity

Step 1: User is Online (Everything Works)



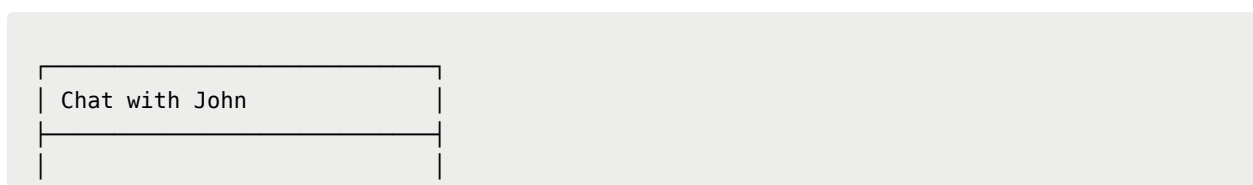
Step 2: Internet Connection is Lost

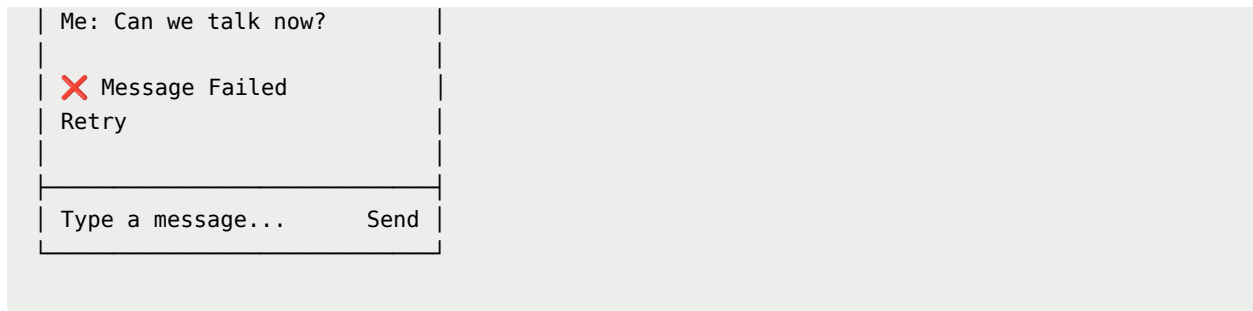


Problem #1

User cannot access conversation history because messages are not stored locally.

Step 3: User Tries to Send a Message

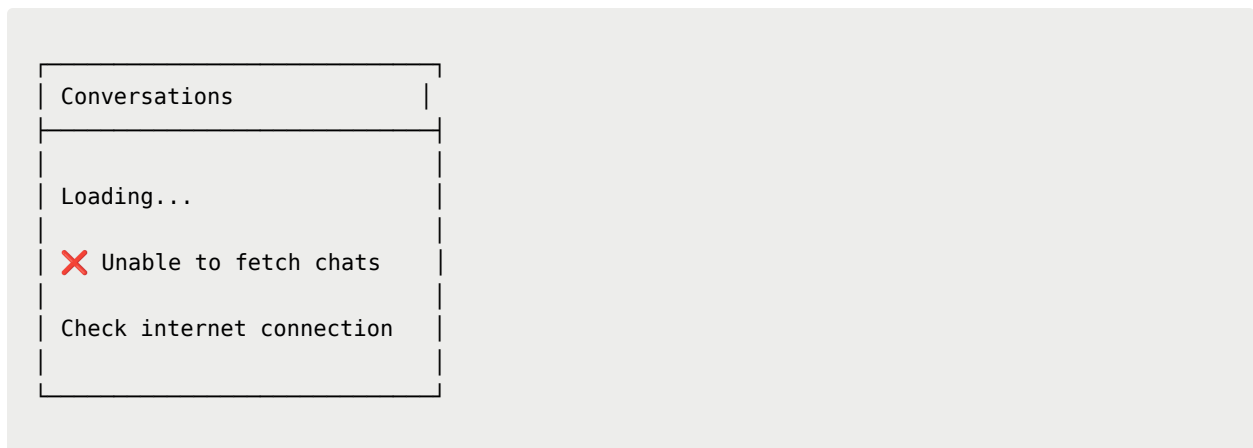




Problem #2

Messages cannot be reliably sent while offline.

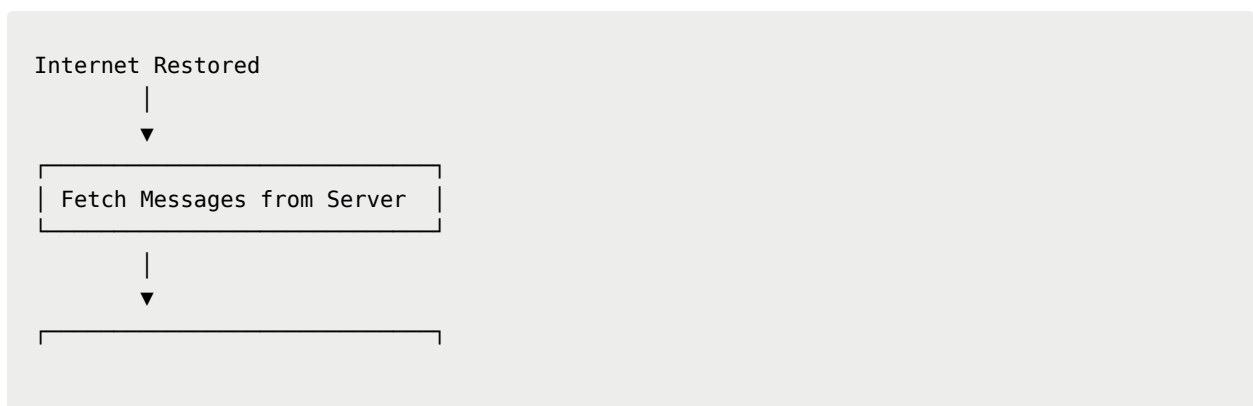
Step 4: User Closes and Reopens App While Offline



Problem #3

User cannot view conversations when there is no internet connection.

Step 5: Internet Returns

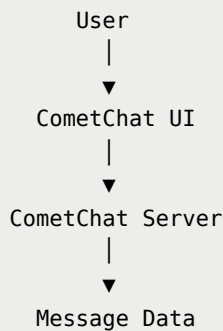


| Load Missed Messages |

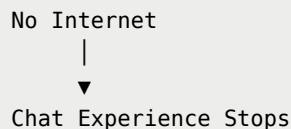
Current CometChat Behavior

- ✓ Messages can be recovered after reconnecting.
- ✗ Conversations are not available offline.
- ✗ No local message storage.
- ✗ No offline message queue.
- ✗ No automatic synchronization layer.

Current Architecture



Dependency

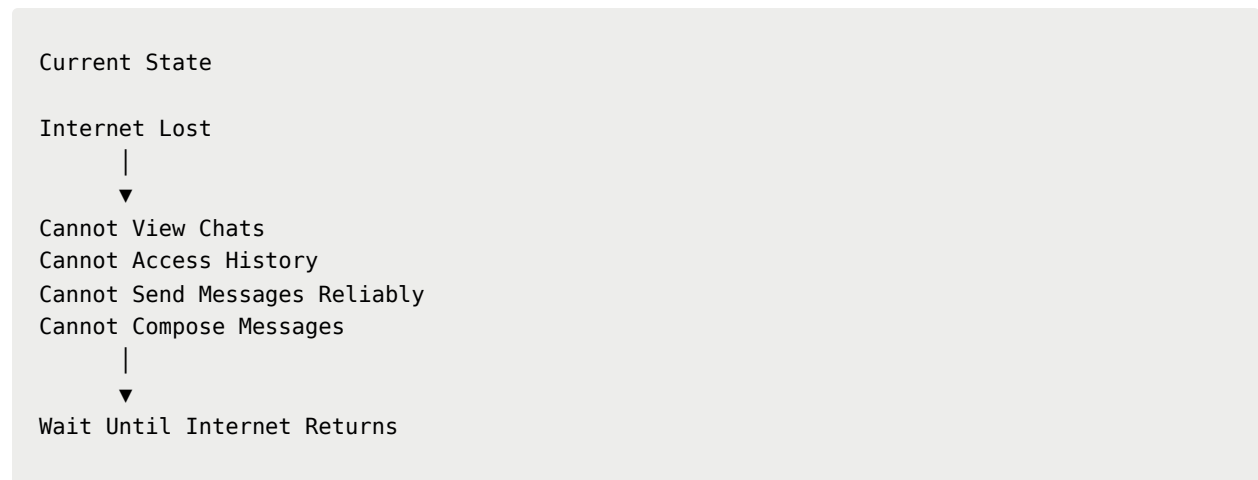


Core Problem Statement

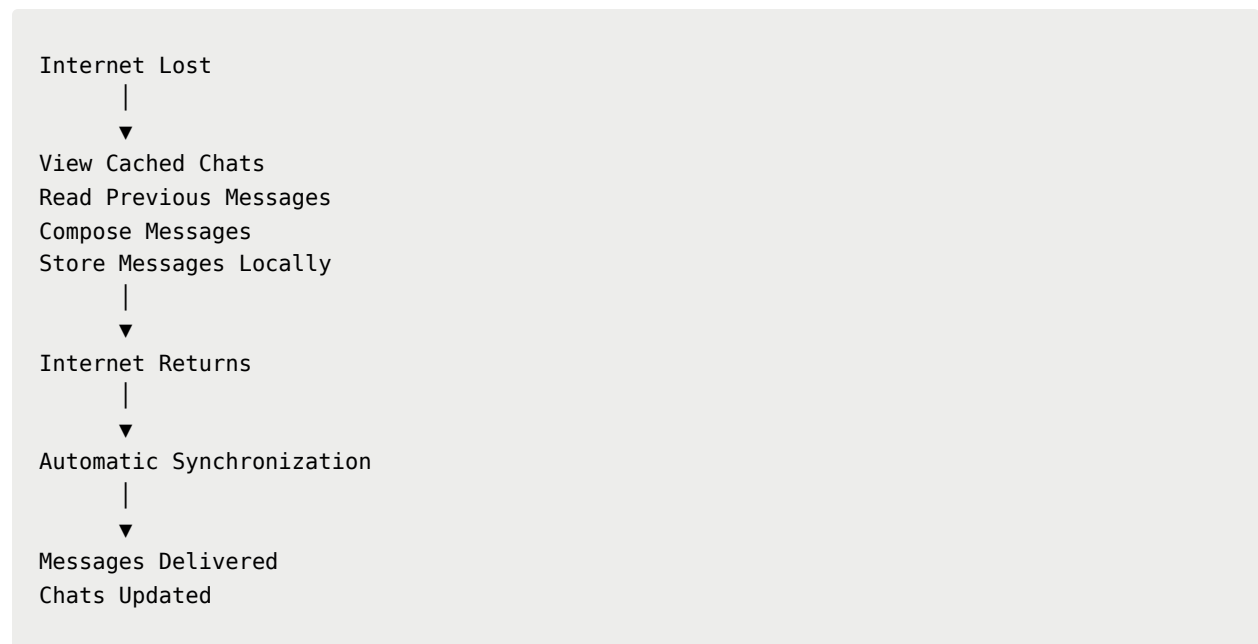
Users lose access to conversations and experience delays when network connectivity is poor or unavailable. While CometChat can recover missed

messages after reconnection, users cannot continue reading chats, accessing conversation history, or reliably sending messages during offline periods because there is no dedicated offline storage and synchronization layer.

What Needs to Be Solved



The proposed Offline Storage & Sync feature should enable:



This wireframe highlights the current user pain points and clearly shows why Offline Storage and Sync is needed in CometChat.

Goal

Enable users to:

- View recent conversations offline
- View cached messages offline
- Compose messages offline
- Automatically sync upon reconnection
- Maintain consistent message state across devices

[link to Product Discovery Doc]

Success Metrics

How will we know this worked? One primary metric, plus a guardrail for what shouldn't get worse.

Category	Metric	Target (CometChat V1)	Why It Matters	Benchmark vs Sendbird
User Experience	Offline Chat Availability	> 99% of previously synced conversations accessible offline	Users can continue viewing chats without internet	Similar to Sendbird Local Cache
User Experience	Chat Screen Load Time (Offline)	< 1 second	Instant access to conversations from local storage	Comparable experience
Messaging Reliability	Offline Message Queue Success Rate	> 99.5%	Ensures queued messages are delivered after reconnect	Expected parity
Messaging Reliability	Message Loss Rate	0%	No messages should be lost during offline-	Critical benchmark

Category	Metric	Target (CometChat V1)	Why It Matters	Benchmark vs Sendbird
			to-online transitions	
Messaging Reliability	Duplicate Message Rate	< 0.1%	Prevent duplicate messages during retries and sync	Comparable to Sendbird's deduplication logic
Synchronization	Sync Completion Success Rate	> 99%	Successful processing of queued actions and missed messages	Competitive parity
Synchronization	Average Reconnect Sync Time	< 5 seconds for normal usage	Users quickly see latest updates after reconnecting	Similar user expectation
Synchronization	Missed Message Recovery Accuracy	100%	All messages received while offline are recovered	Must match Sendbird capability
Synchronization	Read Receipt Sync Accuracy	> 99%	Read states remain consistent across devices	Competitive requirement
Synchronization	Delivery Receipt Sync Accuracy	> 99%	Delivered status accurately synchronized	Competitive requirement
Data Consistency	Conversation Consistency Rate	> 99.9%	Local cache matches server state after sync	Key trust metric
Data Consistency	Conflict Resolution Success Rate	> 99%	Sync conflicts resolved without user intervention	Important for scale

Category	Metric	Target (CometChat V1)	Why It Matters	Benchmark vs Sendbird
Performance	Local Database Read Time	< 200 ms	Fast rendering of conversations and messages	Better perceived performance
Performance	Local Storage Overhead	< 15% app storage growth per active user	Prevent excessive device storage usage	Industry standard
Performance	Sync API Calls Reduction	> 50% reduction vs non-cached implementation	Lower backend load and bandwidth consumption	Major advantage of offline architecture
Platform Stability	App Crash Rate During Sync	< 0.1%	Sync engine should not impact application stability	Critical KPI
Platform Stability	Queue Recovery Success Rate After App Restart	> 99%	Pending messages survive app restarts and crashes	Expected enterprise-grade behavior
Adoption	Offline Feature Usage Rate	Track % users accessing chats offline	Measures real-world usage and value	Product insight metric
Adoption	Customer Support Tickets Related to Message Sync	Reduce by 50%	Indicates improved reliability and trust	Business outcome metric

Assumptions

1. Persistent Local Database is Available

- Each platform (Android, iOS, Web) supports a local storage mechanism (e.g., SQLite, Room, Core Data, IndexedDB) to persist chat data across app

sessions and device restarts.

2. **Network Connectivity Can Be Reliably Detected**

- The application can accurately identify online and offline states and trigger synchronization automatically when connectivity is restored.

3. **Local Storage Capacity is Sufficient**

- Devices have adequate storage to cache conversations, messages, users, groups, and pending actions within defined retention limits and cache management policies.

4. **Single Source of Truth is the Server**

- CometChat backend remains the authoritative source of data, while local storage acts as a cache. Any conflicts during synchronization will be resolved based on server-side state and timestamps.

5. **Message Ordering is Preserved**

- Messages and offline actions queued locally will be synchronized in the same sequence in which they were created to maintain conversation consistency.

6. **Offline Support is Limited to Cached Data**

- Users can only access conversations, messages, users, groups, and members that were previously synchronized and stored locally; new data cannot be discovered while offline.

Technical Scope

What's in v1 and what's intentionally deferred. The Out of Scope list should anticipate questions stakeholders will actually ask.

Components Covered Under Offline Storage & Sync

- **Conversations** – Displays a list of recent one-to-one and group chats.
- **Message Header** – Displays user/group information such as profile picture, name.

- **Message List** – Displays the conversation history, including messages, reactions.
- **Message Composer** – Allows users to send text messages, attachments, mentions (@users), and voice notes.
- **Users** – Displays a list of users.
- **Groups** – Displays a list of groups the user belongs to or can access.
- **Group Members** – Displays the list of members within a selected group.

The first version of Offline Storage & Sync includes:

Local Storage

- Conversation cache
- Message cache
- User directory cache
- Group metadata cache
- Group member cache
- Pending message queue

Offline Messaging

- Read cached conversations
- Read cached messages
- Compose text messages offline
- Queue messages for later delivery
- View pending message status

Synchronization

- Automatic synchronization on reconnect
- Missed message retrieval
- Delivery receipt synchronization

- Read receipt synchronization
- Conversation metadata synchronization
- Group and user synchronization

UI Enhancements

- Offline banner
 - Sync-in-progress indicator
 - Pending message status
 - Failed message status
 - Retry mechanism for failed messages
-

Out of Scope

Offline Voice & Video Calling

- Audio calling functionality
- Video calling functionality

Administrative Operations

- Group creation while offline
- User management actions while offline
- Group management while offline

Full Media Offline Support

- Downloading all media files or high resolutions automatically for offline viewing.

Messages

Reporting a Message

Message Translations

Search Message

Project Phases

How the work breaks into shippable releases; what each phase delivers and what blocks it.

Phase 1 – Core Offline Experience (MVP)

Goal

Enable users to view previously loaded data and continue basic messaging while offline, with automatic synchronization when connectivity returns.

Components Covered

Conversations

- View cached conversation list
- View last message
- View unread count

Message Header

- Display cached user/group information
- Display cached profile pictures and names

Message List

- View cached message history
- Display pending message states

Message Composer

- Send text messages offline
- Queue messages for delivery after reconnect

Users

- View cached user directory

Groups

- View cached group list and metadata

Group Members

- View cached group members
-

Local Storage

- Conversation cache
 - Message cache
 - User directory cache
 - Group metadata cache
 - Group member cache
 - Pending message queue
-

Offline Messaging

- Read cached conversations
 - Read cached messages
 - Compose text messages offline
 - Queue messages for later delivery
 - View pending/sending/failed message states
-

Synchronization

- Automatic sync on reconnect
- Missed message retrieval
- Conversation synchronization
- Delivery receipt synchronization
- Read receipt synchronization

- User synchronization
 - Group synchronization
-

Acceptance Criteria

- ✓ Users can access chats without internet
 - ✓ Users can send text messages offline
 - ✓ Messages automatically sync after reconnect
 - ✓ Conversation list remains updated
 - ✓ No message loss during network interruptions
-

Phase 2 – Advanced Offline Experience

Goal

Expand offline support to rich messaging features and advanced synchronization scenarios.

Message Composer Enhancements

- Attachment support offline
 - Voice note support offline
 - Mentions (@users) support offline
 - Rich media synchronization
-

Message Interactions

- Reactions synchronization
 - Message edit synchronization
 - Message delete synchronization
 - Reply/thread synchronization (if supported)
-

Group Enhancements

- Group membership updates
 - Group setting synchronization
 - Role and permission updates
-

Performance Enhancements

- Cache size management
 - Automatic cache cleanup
 - Large conversation optimization
 - Incremental database updates
-

Analytics & Monitoring

- Sync success/failure tracking
 - Queue monitoring
 - Offline usage metrics
 - Sync latency tracking
-

Out of Scope (Future Releases)

The following capabilities are not included in V1:

- Offline attachment upload
- Offline voice notes
- Offline reactions
- Offline message editing
- Offline message deletion
- Offline thread/reply support
- Offline polls
- Cross-device offline queue synchronization

- End-to-end encrypted offline key management
- Offline group creation
- Offline user creation
- Offline group membership management

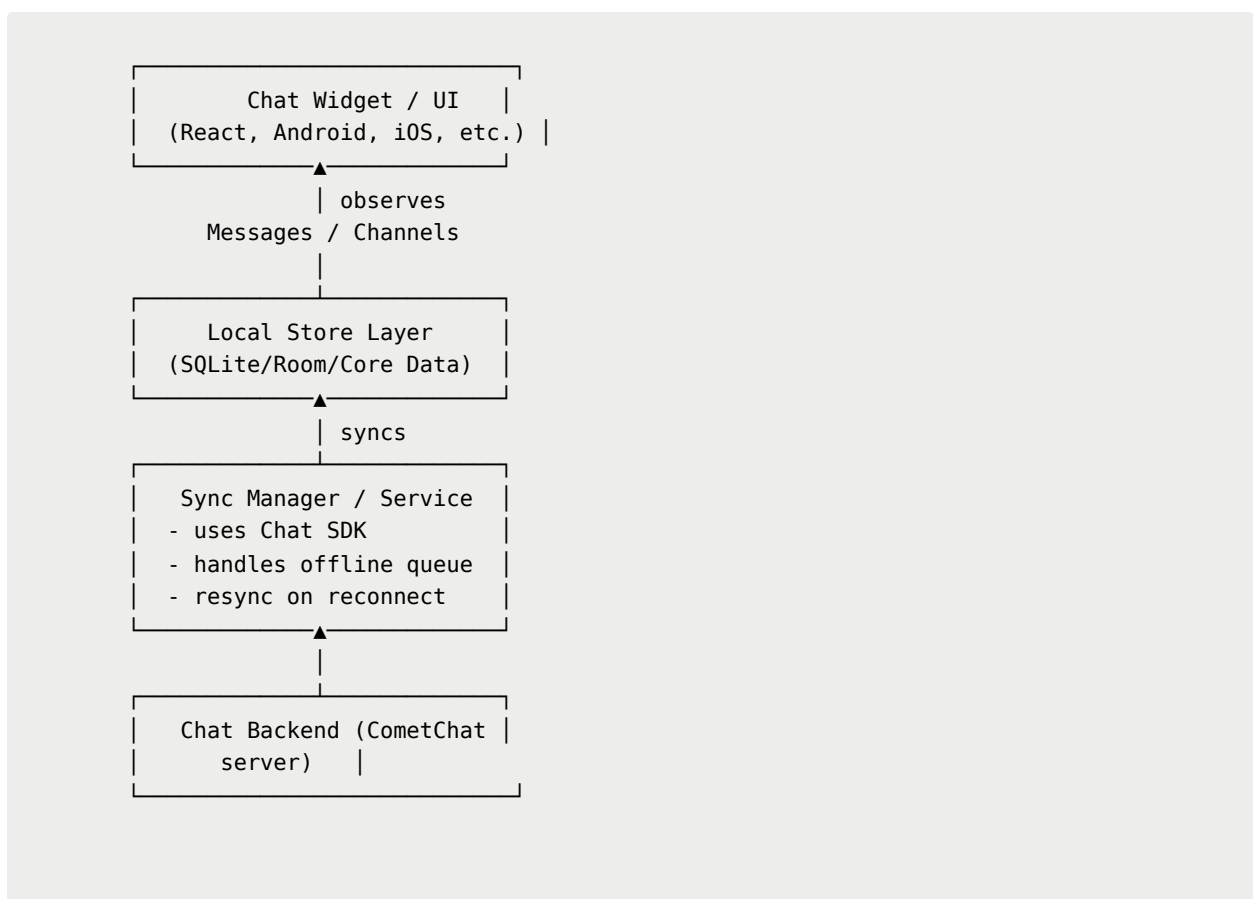
These capabilities can be evaluated after the core offline messaging experience is stable and adopted.

Continue adding phases below using the same structure: phase title, what it delivers, and the work it includes.

Detailed Steps

Screen-by-screen specs for the primary flow. H1 copy, body, behavior, fields, and actions per step.

Technical Diagram:



Step 1: Load Chat Screen

When a user opens a conversation:

1. UI requests messages from the local database.
2. Messages are displayed immediately.
3. No server call is required to render existing messages.
4. User can view previous chats even when offline.

Expected Result:

- Fast chat loading.
 - Offline chat access.
-

Step 2: Store Messages Locally

Every message should be saved in the local database.

Store:

- Conversations
- Messages
- Attachments metadata
- Read receipts
- Delivery status
- Pending offline actions

Message Statuses:

- Pending
- Sending
- Sent

- Delivered
- Read
- Failed

The UI should always read data from the local database.

Step 3: Sending Messages While Offline

When the user sends a message without internet:

1. Save the message in the local database.
2. Assign a temporary local message ID.
3. Mark status as "Pending".
4. Add the message to the Offline Queue.
5. Immediately display the message in the UI.

Expected Result:

- User sees the message instantly.
 - No message is lost.
-

Step 4: Offline Action Queue

The Sync Manager maintains a queue of actions performed while offline.

Supported actions:

- Send Message
- Edit Message
- Delete Message
- Add Reaction
- Mark Message as Read

The queue must preserve execution order.

Example:

1. Send Message A
 2. Send Message B
 3. Add Reaction
 4. Delete Message
-

Step 5: Monitor Network Status

Sync Manager continuously monitors connectivity.

When internet is lost:

- Switch to Offline Mode.
- Continue saving actions locally.

When internet returns:

- Trigger Synchronization Process.
-

Step 6: Synchronize Pending Actions

Upon reconnection:

1. Read all pending actions from the queue.
2. Execute them sequentially using Chat SDK APIs.
3. Receive server response.
4. Update local database.
5. Notify UI automatically.

Expected Result:

- Pending messages become Sent/Delivered.
 - UI reflects latest status.
-

Step 7: Synchronize New Incoming Messages

After reconnecting:

1. Retrieve last successfully synced message ID.
2. Request all messages newer than that ID from the server.
3. Save received messages into local storage.
4. Refresh UI automatically.

Expected Result:

- User receives all missed messages.
-

Step 8: Conversation Synchronization

During sync:

1. Fetch latest conversation list.
2. Update:
 - Last message
 - Unread count
 - Conversation metadata
3. Store updates locally.
4. Refresh conversation screen.

Expected Result:

- Conversation list remains consistent with server.
-

Step 9: Read Receipt Synchronization

When offline:

1. User opens a conversation.

2. Read event is stored locally.
3. Event is added to the offline queue.

When online:

1. Send read receipt to server.
2. Update unread counts.
3. Update local database.

Expected Result:

- Read status remains accurate across devices.
-

Step 10: Attachment Synchronization

While Offline:

1. Save attachment locally.
2. Mark attachment as Pending.

When Online:

1. Upload attachment.
2. Receive attachment URL.
3. Send message containing attachment URL.
4. Update local database.
5. Mark message as Sent.

Expected Result:

- Attachments are automatically delivered after reconnecting.
-

Key Development Principles

Principle 1: Local Database is the Source of Truth

UI should always read data from local storage.
Do not render data directly from server responses.

Principle 2: Update Database Before UI

Every server event should:

Server Response

- Update Local Database
 - UI Automatically Refreshes
-

Principle 3: Queue All Offline Actions

Every action performed without connectivity must be stored and retried automatically.

Principle 4: Incremental Synchronization

Always fetch only new data since the last successful sync.
Avoid full conversation reloads.

Risk Area's

Unresolved decisions and known risks. Pair each with a mitigation, owner, or next step.

1. Message Duplication & Ordering Risk

Risk: During reconnection, the same message may be sent multiple times or appear out of sequence due to retries, network interruptions, or SDK reconnect events.

Why Sendbird Handles This Better:

- Sendbird generates temporary local IDs and maps them to server IDs after successful delivery.

- Messages are synchronized using ordered event streams.
-

2. Large Local Database Performance Risk

Risk: Applications with thousands of conversations and messages may experience:

- Slow startup times
- Increased memory consumption
- Delayed synchronization

Why Sendbird Handles This Better:

- Uses selective caching and background synchronization.
 - Supports efficient local database management and incremental updates.
-

3. Synchronization Reliability Across Platforms

Risk: Android, iOS, and Web handle background execution differently. Synchronization behavior may become inconsistent across platforms.

Examples:

- Android kills background services.
- iOS restricts background execution.
- Browsers clear storage or suspend tabs.

Why Sendbird Handles This Better:

- Provides platform-specific local cache implementations with consistent synchronization mechanisms.
-

4. Increased SDK Complexity & Maintenance Cost

Risk: Offline storage introduces:

- Local database management
- Queue management
- Sync engine maintenance

- Migration support
- Error recovery workflows

This significantly increases SDK complexity and long-term maintenance effort.

Why Sendbird Handles This Better:

- Offline synchronization is deeply integrated into their SDK architecture rather than implemented as an add-on feature.

Summary

Risk Area	Potential Impact	Sendbird Advantage
Message Duplication	Duplicate or missing messages	Temporary IDs & deduplication
Local Storage Scale	Performance degradation	Optimized local cache
Cross-Platform Sync	Inconsistent behavior	Mature platform support
SDK Complexity	Higher development effort	Built-in architecture